

UNIVERSIDAD

DEPARTAMENTO DE SEGURIDAD DE LA INFORMACIÓN

Práctica: Wipe Remoto

Limpieza Remota de Datos Sensibles en Aplicación Móvil

Materia: Seguridad de la Información

Profesor: Jose Alonso Macias Montoya

Alumno: Hugo (IDS-HUGO)

1. Introducción y Justificación

En el contexto de la seguridad en dispositivos móviles, la pérdida o el robo físico de un dispositivo móvil representa una de las mayores amenazas para la confidencialidad de la información del usuario y de las organizaciones. Si un atacante tiene acceso físico al dispositivo, puede intentar extraer datos en caché, tokens de sesión o información confidencial guardada localmente.

El **Wipe Remoto (Limpieza Remota)** es una contramedida crítica que permite a un administrador o al propio usuario enviar un comando a través de internet para borrar inmediatamente todos los datos confidenciales almacenados localmente en la aplicación. Esto previene el acceso no autorizado y mitiga el impacto de un compromiso físico del dispositivo.

Esta práctica implementa un mecanismo de Wipe Remoto en una aplicación móvil Flutter utilizando **Firebase Cloud Messaging (FCM)** como canal de comunicación en tiempo real y **Flutter Secure Storage** para resguardar criptográficamente los datos bajo los esquemas de seguridad del sistema operativo (Keystore en Android y Keychain en iOS).

2. Definición del Almacenamiento Seguro

Se ha integrado un almacenamiento seguro local utilizando la biblioteca `flutter_secure_storage`. En el sistema operativo Android, esta biblioteca hace uso de `EncryptedSharedPreferences` (disponible en API 23 o superior), el cual cifra los pares clave-valor utilizando el algoritmo simétrico **AES-256 GCM** con claves administradas por el **Android Keystore System**.

Para el cumplimiento de la práctica, se han definido y almacenado de forma automática los siguientes 4 campos sensibles:

Campo	Clave de Almacenamiento	Descripción del Riesgo / Justificación
1. USUARIO	sensitive_username	Identidad o nombre del usuario. Permite realizar ataques de phishing o ingeniería social si se ve comprometido.
2. CONTRASEÑA	sensitive_password	Clave secreta en texto plano de prueba. Comprometerla compromete permanentemente la cuenta del usuario en todos los servicios que compartan credenciales.
3. CORREO	sensitive_email	Correo electrónico principal. Sirve como identificador único y vector de ataque para el envío de spam, enlaces maliciosos o recuperación de cuentas.
4. TOKEN DE SESION	sensitive_session_token	Token temporal de autorización. Si un atacante lo obtiene, puede realizar un ataque de secuestro de sesión (session hijacking) y suplantar al usuario sin contraseña.

Asignación Automática de Datos

Estos campos se graban automáticamente en el almacenamiento seguro en el instante en que el usuario completa un proceso exitoso de autenticación (Login) o registro en la aplicación móvil, garantizando la persistencia segura durante la sesión activa.

3. Arquitectura y Funcionamiento del Wipe Remoto

Para establecer el canal de Wipe Remoto específico para cada usuario, la aplicación se

comunica con los servidores de **Firebase Cloud Messaging (FCM)**. El flujo de control técnico se divide en dos fases:

3.1 Obtención del Token FCM y Especificidad del Wipe

Al iniciar la aplicación, el servicio de mensajería obtiene un **Token de Registro FCM** único provisto por Google. Para cumplir con el requisito de que el Wipe sea específico para ese usuario (y no general):

1. El token se muestra en la pantalla de la aplicación y permite enviar la notificación directamente a ese dispositivo.
2. El payload de datos (data payload) del mensaje de FCM incluye un filtro de seguridad adicional mediante el campo `target_email`. La aplicación valida el correo almacenado en el Secure Storage antes de borrar los datos:

```
{  "to":  "TOKEN_FCM_DEL_DISPOSITIVO",  "data":  {    "action":  "wipe",  
  
    "target_email": "demo@demo.com"  }  }
```

3.2 Manejo de Estados de la Aplicación

- **Aplicación en Primer Plano (Foreground):** La aplicación escucha activamente los mensajes de FCM mediante `FirebaseMessaging.onMessage`. Si recibe el comando de Wipe, ejecuta el borrado local inmediato de la sesión y campos sensibles, redirige a la pantalla de Login y muestra una alerta interactiva explicativa al usuario.
- **Aplicación en Segundo Plano / Cerrada (Background/Terminated):** El sistema operativo arranca un isolate de ejecución en segundo plano administrado por `FirebaseMessaging.onBackgroundMessage`. Este proceso inicializa Firebase, abre el Secure Storage directamente e implementa el Wipe de manera asíncrona. Además, escribe la bandera de control `remote_wipe_pending_notification` en true.

Al abrir la aplicación nuevamente, el flujo de inicialización detecta la bandera, fuerza el redireccionamiento al Login y despliega el diálogo informativo sobre la limpieza ejecutada en segundo plano.

4. Implementación y Código Principal

4.1 Clase `FirebaseMessagingService` (`firebase_messaging_service.dart`)

Esta clase maneja los permisos, obtiene el Token FCM y coordina los callbacks de recepción en primer y segundo plano:

```
// Handler global de segundo plano @pragma('vm:entry-point') Future
  _firebaseMessagingBackgroundHandler(RemoteMessage message) async { await
    Firebase.initializeApp(); final data = message.data; if (data['action'] ==
    'wipe') { final store = SecureSessionStore(); final targetEmail =
    data['target_email']; if (targetEmail != null &&
    targetEmail.toString().isNotEmpty) { final sensitiveFields = await
    store.readSensitiveFields(); final userEmail = sensitiveFields['email']; if
    (userEmail != null && userEmail.toLowerCase() !=
    targetEmail.toString().toLowerCase()) { return; // No corresponde a este
    usuario } } await store.clear(); await store.setRemoteWipePending(true); } }
```

4.2 Lógica de Wipe en SessionManager (session_manager.dart)

Coordina la limpieza del estado en memoria de la autenticación y la remoción completa de registros en el almacenamiento seguro:

```
Future remotewipe() async { _stopMonitor(); _auth.logout(); _token = null;
```

```
_startedAt = null; remaining.value = Duration.zero; // Borrar todo el Secure
```

```
Storage (incluyendo campos sensibles) await _store.clear(); // Emitir evento
```

```
para la interfaz en primer plano remotewipeEvent.value = true; }
```

5. Guía de Pruebas y Validación

Para validar de forma sencilla e interactiva el funcionamiento de la práctica tanto para el alumno como para el evaluador:

1. **Iniciar Aplicación:** Ejecutar la aplicación en modo Debug en el dispositivo móvil Android.
2. **Copiar Token FCM:** En la pantalla de Login (o en la Home una vez iniciada la sesión), copiar el FCM Token mostrado en pantalla utilizando el botón incorporado.
3. **Autenticarse:** Iniciar sesión con los datos de prueba preestablecidos (demo@demo.com y clave 123456).
4. **Verificar Almacenamiento:** Confirmar que la tarjeta "Datos Sensibles (Secure Storage)" en la pantalla de inicio muestra los 4 campos (Usuario, Contraseña, Correo y Token de Sesión) con la información correcta.
5. **Prueba de Simulación Local (Recomendada para Demostraciones Rápidas):**
 - Presionar el botón "Simular Wipe" en la tarjeta FCM Token de la pantalla

Home.

- Aceptar la confirmación. La aplicación borrará instantáneamente el almacenamiento seguro, se redireccionará al Login y se desplegará el cuadro de diálogo informando la acción.

6. Prueba de Wipe Remoto Real con Firebase Console:

- Copiar el FCM Token.
- Ir a la Consola de Firebase del proyecto -> Messaging.
- Crear una campaña de tipo "Mensaje de notificación".
- En la sección "Enviar mensaje de prueba", pegar el token FCM.
- En la sección de "Opciones adicionales" (Datos personalizados), agregar los pares clave-valor:
 - Clave: `action` | Valor: `wipe`
 - Clave: `target_email` | Valor: `demo@demo.com`
- Enviar. Si la app está abierta, se limpiará en tiempo real. Si está cerrada, se limpiará en segundo plano y se informará al abrirla de nuevo.

6. Conclusión

La implementación de la contramedida de Wipe Remoto proporciona una capa adicional de protección fundamental en el diseño de aplicaciones móviles seguras. Al combinar Firebase Cloud Messaging con almacenamiento seguro encriptado por hardware (Keystore), se logra un control remoto reactivo eficiente que preserva la privacidad del usuario final frente a escenarios críticos de robo de dispositivos o accesos no autorizados a nivel físico.